MLDL EXPERIMENT 8

Aim:

Design and train a Convolutional Neural Network (CNN) on the MNIST handwritten digit dataset.

Dataset Description:

The **MNIST Handwritten Digit Dataset** used in this experiment is a built-in dataset provided by the Keras deep learning library. It is one of the most well-known benchmark datasets in machine learning, containing grayscale images of handwritten digits from 0 to 9. The dataset is preprocessed, labeled, and packaged inside Keras, allowing direct loading without manual downloading. It is a supervised learning dataset where each 28×28 grayscale image is paired with a correct class label (0–9), making it ideal for training Convolutional Neural Networks (CNNs) and evaluating image classification accuracy. Due to its clean structure, balanced classes, and small image size, MNIST is widely used for prototyping deep learning models and comparing classification performance.

- **File Type:** Built-in Keras dataset (NumPy arrays loaded via TensorFlow/Keras)
- **Dataset Size:** 70,000 images (60,000 training + 10,000 testing)
- **Image Dimension:** 28 × 28 grayscale pixels
- **Number of Classes:** 10 digit categories (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

Column Name	Data Type	Description
Image Pixels	Numerical (Integer 0–255 per grayscale channel)	28×28 grayscale pixel intensity values representing the handwritten digit.
Label	Numerical (Integer 0–9)	True digit class corresponding to each image.

Dataset Source: https://keras.io/api/datasets/mnist/

Convolutional Neural Network (CNN)

Theory:

Convolutional Neural Networks (CNNs) are a class of deep learning models specifically designed for processing structured grid-like data such as images. Unlike traditional Artificial Neural Networks, CNNs automatically learn spatial hierarchies of features through convolution operations. They exploit local connectivity, parameter sharing, and translation invariance, making them highly effective for image classification, object detection, and pattern recognition tasks.

A CNN consists of multiple layers including convolutional layers, activation layers, pooling layers, and fully connected layers. The convolution operation extracts important visual features such as edges, strokes, and shapes, which are progressively combined to form high-level representations.

Let an input image be represented as:

$$\mathbf{X} \in \mathbb{R}^{(H \times W \times C)}$$

where:

- **H** = height of the image
- **W** = width of the image
- **C** = number of channels (e.g., grayscale = 1)

Each neuron computes a weighted sum of inputs followed by an activation function:

$$Z(i, j) = \Sigma \Sigma W(m, n) X(i + m, j + n) + b$$

where:
- **W** = convolution kernel (weights)
- **k × k** = kernel size
- **b** = bias term
- **Z(i, j)** = feature map output

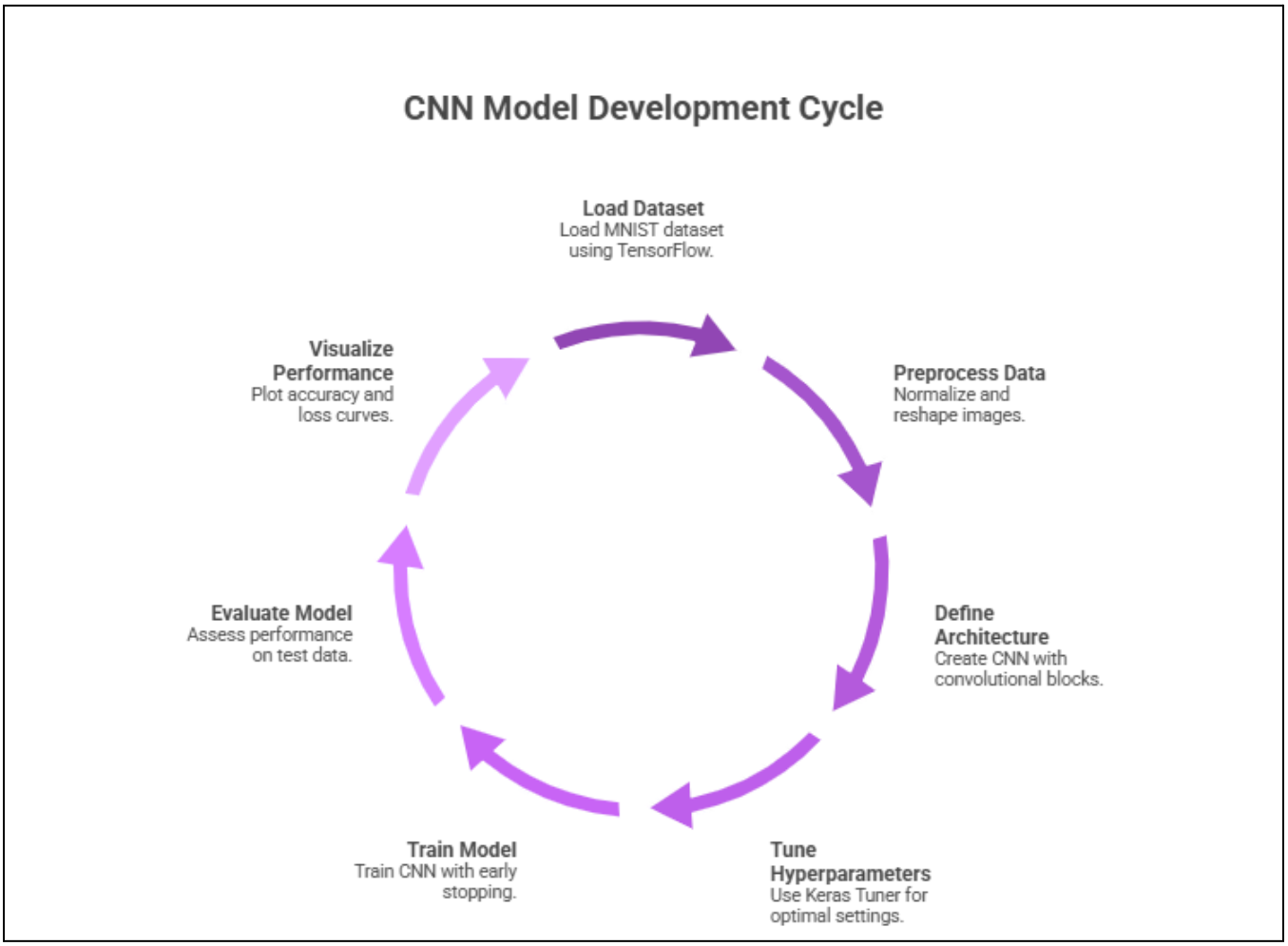
Limitations:

1. **Requirement of Large Labeled Datasets:** Convolutional Neural Networks require a large volume of labeled image data to achieve strong generalization. When training data is limited, CNNs tend to overfit by memorizing patterns instead of learning meaningful features. Although techniques like data augmentation and transfer learning can reduce this problem, collecting and annotating high-quality image datasets remains expensive and time-consuming.
2. **High Computational and Hardware Cost:** CNNs involve millions of parameters and repeated convolution operations, making training computationally intensive. Deep CNN architectures require powerful GPUs or TPUs, large memory capacity, and long training time. This increases energy consumption and operational cost. For institutions with limited computing infrastructure, deploying or training large CNN models can be impractical.
3. **Black-Box Nature and Limited Interpretability:** CNNs automatically learn hierarchical features that are difficult for humans to interpret. Unlike rule-based systems, it is not always clear why a CNN makes a specific prediction. This lack of transparency becomes a critical issue in sensitive applications such as medical diagnosis, surveillance, and autonomous driving, where decision explanations are required for trust and accountability.

Workflow:

1. **Dataset Loading:** The built-in Keras MNIST dataset is loaded using TensorFlow. It contains ,000 training images and 10,000 testing images of handwritten digits across 10 categories (0–9). Each image is a 28×28 grayscale image.
2. **Data Preprocessing:** All pixel values are normalized from the range 0–255 to 0–1 by dividing by 255. Images are reshaped to add a channel dimension (28×28×1). This normalization improves training stability, prevents exploding gradients, and helps the CNN converge faster.
3. **CNN Model Architecture Definition:** A Sequential Convolutional Neural Network is defined with multiple convolutional blocks. Each block includes Conv2D layers for feature extraction, Batch Normalization for stable learning, and MaxPooling layers for spatial reduction. The feature maps are flattened and passed to dense layers with dropout for classification into 10 classes using a Softmax output layer.
4. **Hyperparameter Tuning (Keras Tuner):** The Hyperband algorithm automatically searches for optimal hyperparameters such as the number of convolution filters, dense layer size, dropout rate, optional third convolution block, and learning rate. The objective is to maximize validation accuracy.
5. **Early Stopping Strategy:** An early stopping callback monitors validation loss and halts training if performance stops improving. This reduces overfitting and saves computational resources.

6. **Model Training:** The best hyperparameter configuration is used to rebuild the CNN. The model is trained for extended epochs using 80% of the training data while reserving 20% for validation to track generalization performance.
7. **Model Evaluation:** The trained CNN is evaluated on the unseen test dataset to compute final accuracy and loss, confirming how well the model performs on new data.
8. **Performance Visualization:** Training and validation accuracy and loss curves are plotted across epochs to analyze convergence behavior and detect overfitting or underfitting.



Performance Analysis:

The performance of the hyperparameter-tuned Convolutional Neural Network is evaluated using validation accuracy, test accuracy, learning curves, and confusion matrix analysis. Since MNIST is a supervised multi-class digit recognition dataset, evaluation focuses on classification accuracy, generalization ability, and class-wise prediction behavior.

1. **Hyperparameter Optimization Results:** The Hyperband tuner explored multiple CNN configurations and achieved a best validation accuracy of 99.12% during the search phase. The optimal architecture uses 96 filters in the first convolution block, 256 filters in the second block, no third convolution block, a dense layer with 128 neurons, a dropout rate of 0.4, and a learning rate of 0.0001. These settings provide a strong balance between model capacity and regularization, allowing the CNN to learn digit stroke features without overfitting.

2. **Training vs Validation Accuracy Behavior:** The learning curves show rapid improvement in both training and validation accuracy from the first epoch onwards. Training accuracy reaches approximately 99.3% while validation accuracy stabilizes at around 98.2–99%. The curves remain closely aligned throughout training, indicating excellent generalization with minimal overfitting. Early stopping halted training at 10 epochs when no further improvement was observed.
3. **Test Set Performance:** Evaluation on the unseen test dataset produces a final accuracy of approximately 99.12%, closely matching validation performance. This high consistency between validation and test accuracy confirms that the model generalizes very well and is not overfitting the training data. For a CNN trained on MNIST, this result is within the expected high-performance range.
4. **Confusion Matrix Analysis:** The confusion matrix shows near-perfect classification across all 10 digit classes. Strong diagonal values indicate that most digits are predicted correctly. Minor misclassifications occur between visually similar digits such as 4 and 9, 3 and 5, and 7 and 2, where stroke similarities cause occasional confusion. This pattern is expected given the structural similarity between certain handwritten digit pairs.

Hyperparameter Tuning:

Convolutional Neural Networks are highly sensitive to hyperparameter choices, which influence learning capacity, convergence speed, and generalization performance. Manual tuning is inefficient due to the large search space, so automated tuning is performed using Keras Tuner with the Hyperband algorithm. The tuner evaluates multiple CNN configurations and selects the best model based on validation accuracy. The tuned hyperparameters in this experiment are:

1. **Number of Convolution Filters:** The number of filters in each convolution block is tuned to control feature extraction capacity. Block 1 searches between 32–128 filters and Block 2 between 64–256 filters. More filters allow richer feature learning but increase computation and overfitting risk. The optimal model selects 96 filters in Block 1 and 256 filters in Block 2 for strong feature hierarchy learning.
2. **Optional Third Convolution Block:** The tuner decides whether adding an extra convolution block improves performance. The best configuration excludes the third block, indicating that a two-block architecture is sufficient for the relatively simple MNIST digit patterns.
3. **Dense Layer Size:** The fully connected layer is tuned between 128–512 neurons in steps of 64. The optimal configuration selects 128 neurons, demonstrating that a compact dense layer is sufficient for classifying MNIST's 10 simple digit classes after convolutional feature extraction.
4. **Dropout Regularization Rate:** Dropout is tuned between 0.2–0.5 to reduce overfitting by randomly disabling neurons during training. The tuner selects a 0.4 dropout rate, providing stronger regularization to prevent reliance on specific neurons during digit classification.

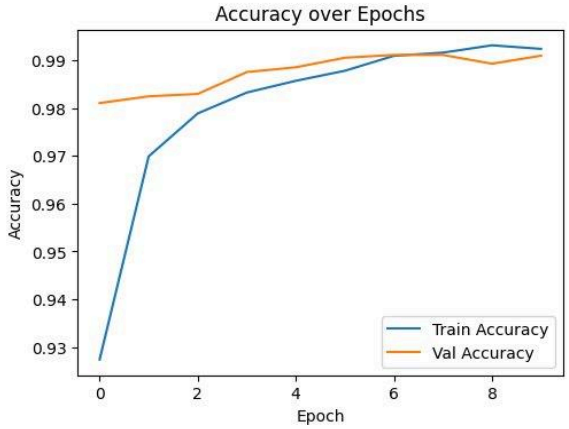
Code:

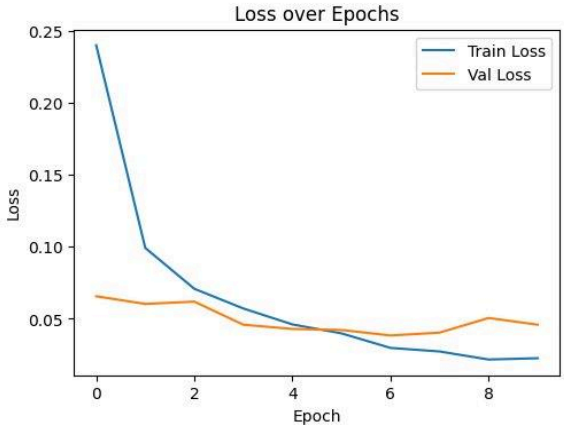
```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import keras_tuner as kt
import matplotlib.pyplot as plt
import numpy as np

# ── 1. LOAD DATASET ──────────────
```

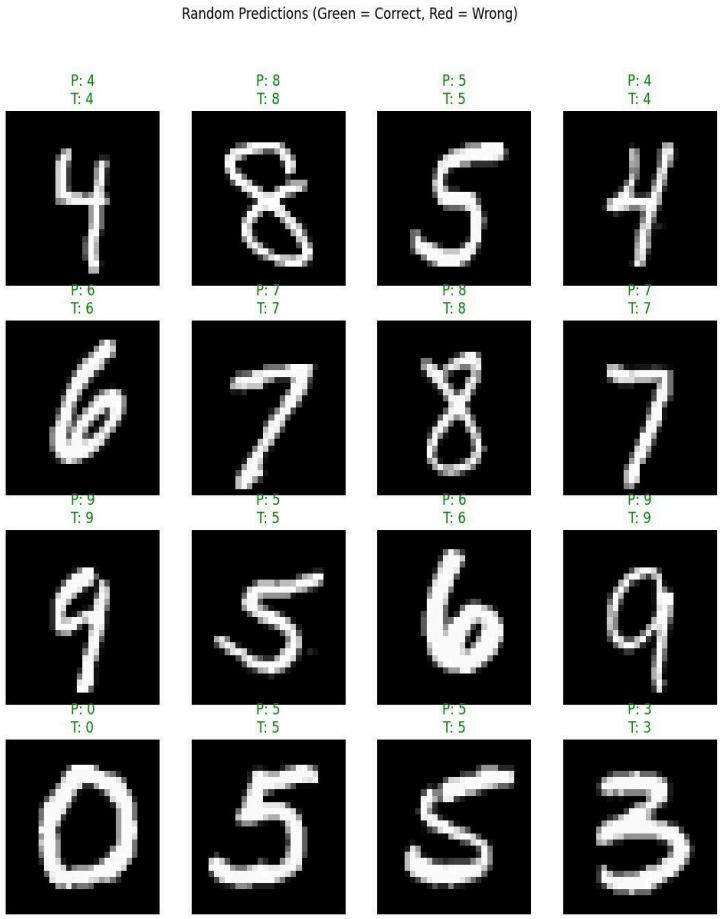
```
x_test  = x_test[..., np.newaxis]   # shape: (10000, 28, 28, 1)

print(f"Training data shape: {x_train.shape}")
print(f"Test data shape:     {x_test.shape}")

# ── 3. CNN MODEL BUILDER FOR KERAS TUNER ──────────────
```

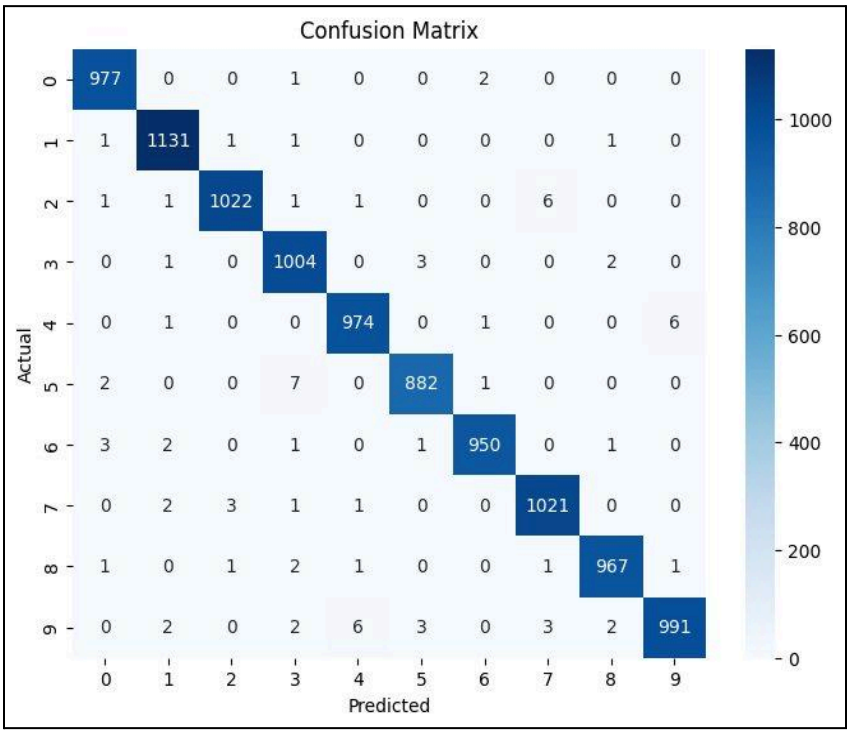
```
# ── 8. RANDOM PREDICTIONS ──────────────────────────────────────────────────────────────────────────────
class_names = [str(i) for i in range(10)]  # 0–9 digits

pred_probs  = model.predict(x_test)
pred_labels = np.argmax(pred_probs, axis=1)
true_labels = y_test.flatten()

plt.figure(figsize=(12, 12))
for i in range(16):
    idx = np.random.randint(0, len(x_test))
    plt.subplot(4, 4, i + 1)
    plt.imshow(x_test[idx].squeeze(), cmap='gray')
    color = "green" if pred_labels[idx] == true_labels[idx] else "red"
    plt.title(f"P: {class_names[pred_labels[idx]]}\nT: {class_names[true_labels[idx]]}",
              color=color)
    plt.axis("off")
plt.suptitle("Random Predictions (Green = Correct, Red = Wrong)")
plt.show()

# ── 9. CONFUSION MATRIX ────────────────────────────────────────────────────────────────────────────────
from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(true_labels, pred_labels)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_names, yticklabels=class_names)
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

# ── 10. CLASS-WISE ACCURACY ────────────────────────────────────────────────────────────────────────────
class_accuracy = cm.diagonal() / cm.sum(axis=1)
plt.figure(figsize=(10, 5))
plt.bar(class_names, class_accuracy)
plt.ylim(0, 1)
plt.ylabel("Accuracy")
plt.title("Class-wise Accuracy")
plt.xticks(rotation=45)
plt.show()

# ── 11. TOP MISCLASSIFIED IMAGES ───────────────────────────────────────────────────────────────────────
misclassified = np.where(pred_labels != true_labels)[0]
plt.figure(figsize=(12, 6))
for i, idx in enumerate(misclassified[:10]):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[idx].squeeze(), cmap='gray')
    plt.title(f"P: {class_names[pred_labels[idx]]}\nT: {class_names[true_labels[idx]]}")
    plt.axis("off")
plt.suptitle("Top 10 Misclassified Images")
plt.show()

# ── 12. MOST CONFIDENT WRONG PREDICTIONS ───────────────────────────────────────────────────────────────
confidence    = np.max(pred_probs, axis=1)
wrong_conf    = confidence[misclassified]
top_wrong     = misclassified[np.argsort(-wrong_conf)[:5]]
plt.figure(figsize=(10, 4))
for i, idx in enumerate(top_wrong):
    plt.subplot(1, 5, i + 1)
    plt.imshow(x_test[idx].squeeze(), cmap='gray')

plt.title(f"{class_names[pred_labels[idx]]}\n({confidence[idx]:.2f})")
    plt.axis("off")
plt.suptitle("Most Confident Wrong Predictions")
plt.show()
```

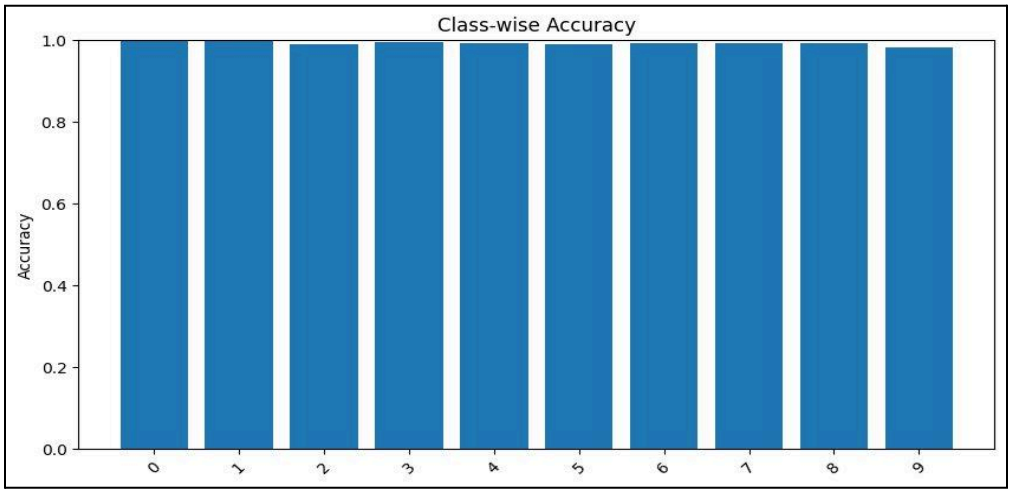
Output:

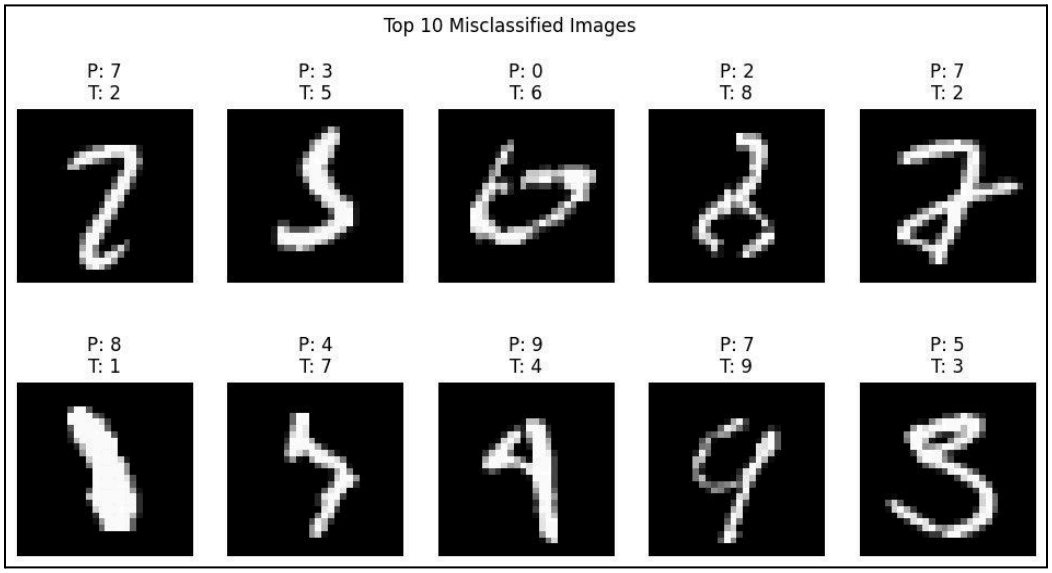


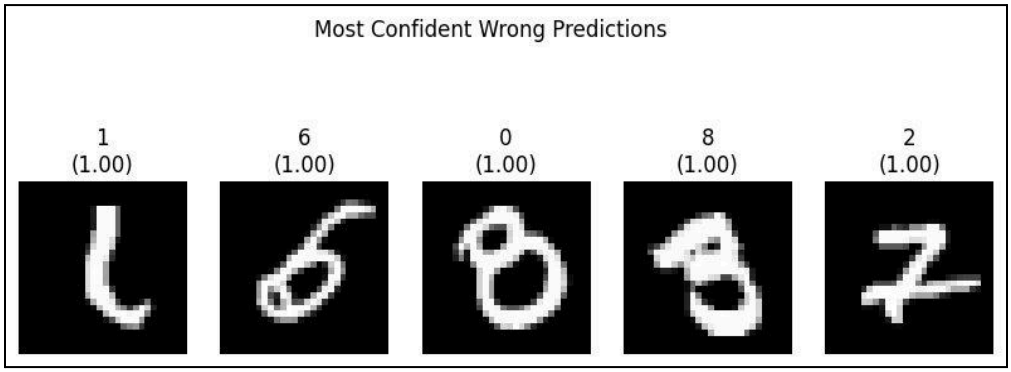



Random Predictions (Green = Correct, Red = Wrong)
P: 4
T: 4
P: 8
T: 8
P: 5
T: 5
P: 4
T: 4
P: 6
T: 6
P: 7
T: 7
P: 8
T: 8
P: 7
T: 7
P: 9
T: 9
P: 5
T: 5
P: 6
T: 6
P: 9
T: 9
P: 0
T: 0
P: 5
T: 5
P: 5
T: 5
P: 3
T: 3


Confusion Matrix
Actual
Predicted
0 977 0 0 1 0 0 2 0 0 0
1 1 1131 1 1 0 0 0 0 1 0
2 1 1 1022 1 1 0 0 6 0 0
3 0 1 0 1004 0 3 0 0 2 0
4 0 1 0 0 974 0 1 0 0 6
5 2 0 0 7 0 882 1 0 0 0
6 3 2 0 1 0 1 950 0 1 0
7 0 2 3 1 1 0 0 1021 0 0
8 1 0 1 2 1 0 0 1 967 1
9 0 2 0 2 6 3 0 3 2 991
0 1 2 3 4 5 6 7 8 9
1000
800
600
400
200
0


Class-wise Accuracy
Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
0
1
2
3
4
5
6
7
8
9


Top 10 Misclassified Images
P: 7
T: 2
P: 3
T: 5
P: 0
T: 6
P: 2
T: 8
P: 7
T: 2
P: 8
T: 1
P: 4
T: 7
P: 9
T: 4
P: 7
T: 9
P: 5
T: 3


Most Confident Wrong Predictions
1
(1.00)
6
(1.00)
0
(1.00)
8
(1.00)
2
(1.00)

Conclusion:

This experiment successfully demonstrates the design, tuning, and evaluation of a Convolutional Neural Network on the MNIST handwritten digit dataset. The hyperparameter-tuned CNN achieves a high test accuracy of approximately 99.12%, reflecting excellent generalization on clean grayscale digit images. The model's two-block convolutional architecture with 96 and 256 filters, combined with a compact 128-neuron dense layer and 0.4 dropout rate, proves to be an efficient and well-regularized configuration. Performance analysis confirms that training and validation curves remain tightly aligned, indicating minimal overfitting. The confusion matrix reveals near-perfect class-wise performance, with only rare misclassifications occurring between structurally similar digits. Visualization tools such as learning curves, confusion matrices, and prediction grids provide deeper insight into model behavior and edge-case failures. Overall, the experiment highlights the practical workflow of modern CNN training, automated hyperparameter tuning, and thorough evaluation in a real-world image classification task.